

btc_consensus_

Bobby Cupps

May 2026

1 Introduction

The bitcoin protocol was introduced by Satoshi Nakamoto in 2008. It creates a decentralized, replicated ledger of transactions and allows for spending and mining of btc in the network. The interesting piece to me as it relates to CS2620 is the failure tolerance. While Fisher-Lynch-Patterson proved without failure detectors, it was impossible to achieve consensus under some constraints, and various systems we explored up-ed that fault tolerance, it was always by smart message-passing, leadership handoffs, replication, and failure recovery.

The bitcoin protocol also has an algorithm it runs, but it achieves resistance to failure via a *totally* different mechanism. Bitcoin dumps network semantics, leadership, and explicit commits and commit suggestions to the side and instead puts forward an economic incentive for all participants to participate in the network according to the outlined protocol. If this occurs, and no one node is too powerful, in general the network is safe. This writeup aims to probe that assumption of safety even under harmless looking conditions, and partially relate the two modes of dealing with failure: one focused on system design (call this the google way) the second based on economic incentives (call this money-inspired).

2 Bitcoin Explanation

The current protocol for Bitcoin can be found [here](#). Around the world, servers have downloaded this protocol and are currently running it, contributing the mining of additional bitcoin, validation of ongoing transactions, and security of the btc as a store of value across the network.

2.1 UTXO

Unspent Transaction Outputs or UTXO are the amounts, in this case of Bitcoins, left in each users balance at any moment in time. Therefore, with a given valid log of all transactions in time, it can be determined exactly the UTXO that correspond to each user. Then, the power of bitcoin is the peer-to-peer trustless

secure management of the log that maps UTXOs to their users or nodes in the network.

2.2 The Protocol

We will call each node in the bitcoin network a "full node". Among these full nodes, we will differentiate between miners: the full-nodes that are engaged in mining, and non-mining full nodes.

2.2.1 Responsibility of a full node

A full node is responsible for forwarding messages it receives to other nodes it is connected to, and updating its view of the current longest *chain*. It also can submit its own transactions at any time and has a private and public key to facilitate recognition of ownership of Unspent Transaction Outputs.

2.2.2 Responsibility of a mining full node (miner)

Beyond its responsibilities as a full node, a miner seeks to wrap transactions into blocks and add those blocks to the chain. At any moment in time, there is a global mempool, which should not vary drastically from any local view of the mempool. In this mempool are transactions waiting to be verified and added to the chain, so that the corresponding Unspent Transaction Outputs are transferred from the payer to the payee.

There are fees associated with every transaction (a small percentage extra that the payer includes to incentivize verification of the transaction). Miners then assemble a block, which consists of valid transactions (valid according to the rest of the transaction log (chain) it has access to i.e. no double spends, sufficient balances etc.) and can add their block to the chain when they find an SHA256 hash value with which to sign off on the block (proving they did significant work to verify these transactions). Once they find a valid hash, they can add their block to the end of the chain.

But, during mining, miners must be mining at the head of some given chain (adding on to the most recently computed block of that chain). If, at any point they hear about the existence of another chain, longer than the one they are currently mining on, or they hear that a different miner has already extended the chain they were mining on, they abandon their current block, and go begin mining on the head of that chain.

2.3 Evolution of the Chain

At every moment in time, the longest chain in existence is the *cannonical chain*. Outside of that chain there is no value, because that chain records all transactions and allocations of UTXO. Hence, all parties are incentivized to mine on the longest chain at all times. However, there may be branching events.

- A Race-Based Branching event occurs when two or more miners create a fork in a chain where each adds their own new head to the preexisting chain before hearing about the other's newly mined block
- A Partition-Based branching event occurs when, within the network of all full nodes, at least two subsets are fully separated. Then, their chains will evolve independently until the partition is fixed. When that happens, the longest chain will become the *canonical chain*.

Since the first transaction in the bitcoin systems created just one head, there have been many branching events, but they diverge and coalesce again into one chain because all work not done on the main chain is wasted.

During a branching event, only a small suffix of the blocks on the chain will differ (under non-attack situations). This is due to the fact that the aggregate compute power of the network produces blocks faster than any small pool of miners, so once a secondary chain falls behind in length significantly, probabilistically, it will never catch up. But, short term variations in the last 1-7 blocks on the chain do occur. The nature of this variation means that once blocks are "deep enough" on a chain (are not within the most recent 5 blocks) and that chain is still being mined on, those transactions are safe and fixed never to be changed again.

3 Known Attacks

There are known vulnerabilities in Bitcoin's architecture, but most assume undermining the very principles. Before discussing the simulator and simulated attacks on the toy network, a couple known attacks are presented.

3.1 51% attack

In section 2.3, it is mentioned that the chain evolves faster than any one actor or pool of colluding actor. This is what buries transactions safely and what forces all agents to contribute to what they believe is the main chain. Although it undermines the essence of what btc is built on to have one entity control greater than 51% of the chain, if it were to happen, btc would cease to be a decentrally regulated token as the entity with 51% could rewrite all blocks and write all blocks for the future if given enough time.

This is not an attack we need to test, probabilistically the system collapses a.s.

3.2 Eclipse Attack

This is a more network based attack - less catastrophic than the 51% attack. This occurs when a set of attackers can isolate partially or fully some node and then show it a chain that does not align with the real chain. Then, the attacking nodes can trade UTXO to the isolated node for something else, rewrite

the blockchain (at their own pace because they are not racing anyone) and then show the isolated node a new chain that recredits their transaction and does it or another "double spend" again.

Although this is a more interesting attack, it rests on simply whether the attack can force a transaction onto the node being attacked before it realizes the real chain. While the attacker is determining the attacked node's view, it is likely restricted to doing useless work and cannot mine any valuable blocks.

3.3 Selfish-Mine

The selfish mine strategy is laid out in the following paper. In general, colluding selfish nodes do not share the completion of blocks for as long as possible in order to prevent the rest of the network from doing useful work. In this way, they increase their share of the generated revenue.

Because the colluding nodes control less than 50% of the compute, with high probability when they go ahead, they will be caught up to. When this happens, the paper suggests that the colluding nodes can have sentinels distributed throughout the network to forward the "evil" version of the chain before the main chain is propagated. This way, the colluding nodes get the best of both worlds: if they find another block (to go two ahead) then they can invalidate the work of the main chain whenever it finds the next one, or if the main group is the next one to find a block then not all is lost because the colluders reveal their block and get a significant fraction of the network to work on their revealed head.

The paper comes up with the conclusion that even a group that controls a negligible portion of the compute can gain an outsized reward following this strategy.

3.3.1 Our Implementation

This seemed a non-binary failure worth investigating. So, Claude set up a sim that we evolve in a couple ways after v0 allowed for super fast mining, alert forwarding, and therefore chain formation on whatever the longest chain at the time was. The critical parameter I found to be suspicious was γ in their paper, or the percentage of total nodes that would mine on the colluding nodes' head. This was the case because in order to have high gamma, a colluding node must receive the real chain in its propagation, start the propagation of the dishonest chain, and beat the propagation of the real chain to a fraction γ of nodes.

However, my results reflected that even without high gamma, the selfish mine strategy does not work at as low compute power as they suggested. When the selfish miners mine a block, they risk losing any reward by withholding it and not adding to the main chain with hopes that they may increase their reward if they mine 2 or more in a row or their chain gets picked up. Empirically, Table 1 6 shows that this risk does not pay off (at least in my simulator) until significant portions of the overall compute are controlled by the selfish nodes.

4 Economic Breakdown (Network Structure)

In the second evolution of my simulator, I tested varying graph structures to see whether the selfish mining strategy would be more appealing. As might be intuitive from the compute result (that as compute increases it pays more and more to be selfish), as centrality in the graph increases for the selfish component, their relative rewards go up. Hence, miners may be more incentivized to act selfishly when they have the access in the graph to execute this strategy.

5 Economic Breakdown (Fee Structure)

I am absolutely fascinated by the way that blockchain creates a replicated log (albeit with less strict guarantees than the 2620 systems) using economic incentives.

The most obvious way that the economic incentives of bitcoin will continue to change are the market price and the fee vs. subsidies. (how much you are paid automatically to mine a block vs. how much of your reward comes from transaction fees). I will continue to look into this because both in my work and in the world, things that stem from consensus in distributed computing will be incredibly valuable and awesome to build and watch.

6 Tables

Table 1: Selfish-mining performance by selfish hashpower. Each row runs 20 seeds with horizon 5000 for length of chain. Relative revenue is the selfish revenue share divided by α ; values above 1.0 indicate that selfish mining outperformed. Stale rate $\equiv$ blocks generated not on final chain

α	Selfish revenue share	Relative revenue	Std. error	Stale rate
0.05	0.009	0.184	0.010	0.052
0.10	0.035	0.351	0.009	0.090
0.15	0.077	0.516	0.010	0.124
0.20	0.130	0.650	0.010	0.154
0.25	0.196	0.785	0.010	0.185
0.30	0.278	0.925	0.009	0.218
0.35	0.368	1.051	0.008	0.254
0.40	0.491	1.227	0.011	0.301
0.45	0.656	1.458	0.017	0.371

Table 2: Effect of public network topology on selfish-mining performance at $\alpha = 0.35$. Each row runs 20 seeds with chain length horizon 5000

Topology	Parameter	Selfish revenue share	Relative revenue	Std. error	Stale rate
Complete graph	–	0.364	1.040	0.007	0.249
Ring	–	0.388	1.109	0.013	0.259
Random regular	$k = 4$	0.370	1.056	0.008	0.253
Two-community bridge	2 bridges	0.391	1.118	0.007	0.253
Hub-spoke	selfish hub	0.453	1.294	0.006	0.253

Table 3: Main experimental takeaways from v1 and v2

Criterion	Result
First tested profitable α on complete graph	$\alpha = 0.35$
Best tested v2 topology	Hub-spoke with selfish pool as hub
Highest observed relative revenue	1.294
Highest observed stale rate	Ring topology, 0.259